MaCliCo Framework

Rosario Pio Gnazzo *rosariopiognazzo@gmail.com*
Department of Computer Science
University of Salerno

Abstract

MaCliCo (Hypergraph Construction via Maximal Clique based Set Covering), an integer linear programming framework that generalizes a weighted graph into a hypergraph. The goal is to capture the topological relationships among nodes by covering all vertices with as few hyperedges as possible, while keeping the construction robust: no node should appear in too many hyperedges, and each hyperedge should be internally cohesive. MaCliCo proceeds in three sequential steps. First, all maximal cliques of the input graph are enumerated using the Bron–Kerbosch algorithm, in its pivoting and degeneracy-ordering variants. Second, each clique is assigned a normalized cohesion score computed from the weights of its internal edges. Third, a set-cover variant selects the subset of cliques that covers every vertex, jointly penalizing the maximum node redundancy and the number of selected cliques, and rewarding the total internal cohesion. The three objectives are controlled by tunable parameters (α, β, γ). We derive sufficient conditions on these parameters that enforce a lexicographic priority among the objectives, such as redundancy first, then number of hyperedges, then cohesion, or alternatively number of hyperedges first, and provide closed-form choices that guarantee the desired behavior. The selected cliques form the hyperedges of the resulting hypergraph.

1 Introduction

Let be $G = \{V, E_g\}$ a graph and $H = \{V, E_h\}$ a hypergraph where $V = \{v_1, v_2, \ldots, v_n\}$ is the set of vertex v_i and $E_h = \{e_1, e_2, \ldots, e_m\}$ is the set of hyperedges e_j such that $e_j \subseteq V$, while $E_g = \{(v_i, v_j) : \ v_i, v_j \in V \text{ con } i \neq j\}$ is the set of edge between pair of nodes of V.
MaCliCo (Hypergraph Construction via Maximal Clique based Set Covering.) is a PLI framework that takes in input the weighted graph (G) and return in output its generalization as hypergraph (H) minimizing *maximum redundancy* [1] and maximizing *internal cohesion* [2]. The idea behind the framework is that a hypergraph constructed from a graph should capture the topological relationships between nodes, ensuring that all vertices are covered using the smallest number of hyperedges. To do that in a robust way, the following framework is build to avoid that a node is present in many hyperedges and obtain the best internal correlation between the hyperedges.

MaCliCo is based on three sequential operations, the first one is the *extraction of all maximal cliques from the graph* to obtain a collection of subset $\{C_i\}_{i=1}^{k}$. The second operation consists in *weighting the subsets* using the weights of the edges of the nodes internal to each subset. The final and third operation is *extract the minimum number of subsets that cover all vertices*, optimizing redundancy and correlation as described before. At the end, each of these optimal subset C_j^* rappresents the hyperedge of the hypergraph, so the whole set $\{C_j^*\}_{j=1}^{d}$ where $d < k$, constitutes the hypergraph (H).

[1] Given an hypergraph $H = \{V, E\}$, where $V = \{v_1, v_2, \ldots, v_n\}$ is the set of vertex v_i and $E = \{e_1, e_2, \ldots, e_m\}$ is the set of hyperedges e_j. Each v_i can be present in multiple hyperedges e_j, so if $|v_i|$ is the number of e_j covering the vertex v_i, the maximum redundancy is $\max\{|v_i| \ , \forall v_i \in V\}$

[2] Since the hypergraph $H = \{V, E_h\}$ is constructed from a weighted graph $G = \{V, E_g\}$, each hyperedge $e_j \in E_h$ will have an associated score (λ_j) calculated from the weights of the edges between the vertices within it $(w_{ij} \geq 0, \ \forall (i, j) \in E_g)$. The score (λ_j) therefore represents the internal cohesion of the hyperedge e_j, so maximizing the internal cohesion of the hyperedge H means to $\max\{\sum_{\forall e_j \in E_h} \lambda_j\}$

1.1 Maximal Cliques Problem

List all the maximal cliques in an undirected graph is a common computational problem in computer science. There are several results, but for this application, MaCliCo uses the *Bron–Kerbosch algorithm*. Although no polynomial time algorithm is known for this problem, the Bron–Kerbosch algorithm can be used to list all maximal cliques in worst-case optimal time, and it is also possible to list them in polynomial time per clique. There are multiple implementations of the algorithm, however we will focus on two in particular.

1.1.1 Bron–Kerbosch algorithm with Pivoting

The basic form of the algorithm work recursively on three disjoint sets of vertices R, P, and X, it finds the maximal cliques that include all of the vertices in R, some of the vertices in P, and none of the vertices in X. This version is inefficient in the case of graphs with many non-maximal cliques: it makes a recursive call for every clique, maximal or not. To save time and allow the algorithm to backtrack more quickly in branches of the search that contain no maximal cliques, Bron and Kerbosch introduced a variant of the algorithm involving a *pivot vertex* (v^*) chosen from P. Then, neighbors of that pivot element are not recursively tested. Any maximal clique potentially found in tests of neighbors of v^* would also be found when testing v^* or one of its non-neighbors, as at least one of these will always be a part of that maximal clique. Otherwise, only neighbors of v^* would be part of that maximal clique, allowing it to be augmented by adding v^* to it, which contradicts that clique being maximal. Therefore, only v^* and its non-neighbors need to be tested as the choices for the vertex v_i that is added to R in each recursive call to the algorithm.

Algorithm 1 Bron–Kerbosch algorithm with Pivoting

```
1:  procedure BronKerboschPivoting(R, P, X)
2:      if P and X are both empty then
3:          return R as a maximal clique
4:      end if
5:      choose a pivot vertex v* in P ∪ X
6:      for vertex v_i in P \ N(v*) do
7:          BronKerboschPivoting(R ∪ {v_i}, P ∩ N(v_i), X ∩ N(v_i))
8:          P ← P \ {v_i}
9:          X ← X ∪ {v_i}
10:     end for
11: end procedure
```

1.1.2 Bron–Kerbosch algorithm with Vertex Ordering

An alternative method for improving the basic form of the Bron–Kerbosch algorithm involves forgoing pivoting at the outermost level of recursion, and instead choosing the ordering of the recursive calls carefully in order to minimize the sizes of the sets P of candidate vertices within each recursive call. The degeneracy of a graph G is the smallest number d such that every subgraph of G has a vertex with degree d or less. Every graph has a degeneracy ordering, an *ordering of the vertices* such that each vertex has d or fewer neighbors that come later in the ordering; a degeneracy ordering may be found in linear time by repeatedly selecting the vertex of minimum degree among the remaining vertices. If the order of the vertices v_i that the Bron–Kerbosch algorithm loops through is a degeneracy ordering, then the set P of candidate vertices in each call (the neighbors of v_i that are later in the ordering) will be guaranteed to have size at most d. The set X of excluded vertices will consist of all earlier neighbors of v_i, and may be much larger than d.

1.2 Weighting Subsets

After the first operation 1.1, we obtain a list of all maximal cliques $MC = \{C_J\}_{J=1}^{k}$ extract from the graph $G = \{V, E_g\}$, where C_J is a clique, so $C_J = \{V_J, E_J\}$ with $V_J \subseteq V$ and $E_J \subseteq E_g$. Since the graph was weighted, $w_{ij} \geq 0 \quad \forall (v_i, v_j) \in E_g$, for each pair of nodes $(v_i, v_j) \in E_J$ of some $C_J \in MC$, there is the

Algorithm 2 Bron–Kerbosch algorithm with Vertex Ordering

```
procedure BronKerboschVertexOrdering(G)
    P ← V(G)
    R, X ← empty
    for vertex v_i in a degeneracy ordering of G do
        BronKerboschPivoting({v_i}, P ∩ N(v_i), X ∩ N(v_i))
        P ← P \ {v_i}
        X ← X ∪ {v_i}
    end for
end procedure
```

corrisponding weight w_{ij}. So, we using these internal weights to calculate a score λ_J to rappresent the *internal cohesion* of the each $C_J \in MC$.

$$\text{Internal cohesion of each clique} \qquad z_J = \frac{\sum_{(v_i,v_j)} w_{ij}}{\binom{|V_J|}{2}} \qquad \forall (v_i, v_j) \in E_J, \quad \forall C_J \in MC \tag{1}$$

$$\tag{2}$$

$$\text{Standardization} \qquad \lambda_J = \frac{z_J}{\sum_{j=1}^{k} z_j} \in (0,1) \qquad \forall C_J \in MC \tag{3}$$

$$\sum_{j=1}^{k} \lambda_j = 1 \tag{4}$$

1.3 Clique-based Set Cover Problem

The final part of MaCliCo takes in input the collection of weighted subsets $MC = \{C_i\}_{i=1}^{k}$ and the goal is to find the minimum number of weighted subsets that cover all the nodes, minimizing redundancy and maximizing the sum of internal cohesions. This optimization problem is described by the following variant of Set Cover Problem.

$$\min_{y_j,\, R} : \quad \alpha R + \beta \sum_{j=1}^{k} y_j - \gamma \sum_{j=1}^{k} y_j \lambda_j$$

$$\text{s.t.} \quad \sum_{j=1}^{k} a_{ij} y_j \geq 1 \qquad \forall\, v_i \in V \tag{5}$$

$$\sum_{j=1}^{k} a_{ij} y_j \leq R \qquad \forall\, v_i \in V \tag{6}$$

$$y_j \in \{0,1\} \qquad \forall\, j = 1, \ldots, k \tag{7}$$

$$a_{ij} \in \{0,1\} \qquad \forall\, i, j \tag{8}$$

$$R \geq 1 \quad \alpha, \beta, \gamma \geq 0$$

Where the variables are

$$y_i = \begin{cases} 1, & \text{if subset } C_i \in MC \text{ is used} \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$a_{ij} = \begin{cases} 1, & \text{if node } v_i \in C_j \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

$$R: \text{ rapresents the maximum redundancy} \tag{11}$$

$$\alpha, \beta, \gamma : \text{parameters that tune the optimization behavior} \tag{12}$$

1.4 Behavior analysis

The model 1.2 is represented in a general formulation, but depending on the type of behavior we want it to assume during the optimization process, it is necessary to choose the parameters α, β, γ appropriately. The behavior depends on the context in which you operate. For example, in some cases you might want:

1. Optimize the minimization of maximum redundancy
2. Then, while maintaining the same redundancy, minimize the number of subsets to activate
3. Finally, maximize the consistency between the minimum subsets.

This implies that it is necessary to choose α, β, γ so that the more important term always dominates the maximum possible gain of the less important terms, based on the desired behavior. Given the behavior of the example, the sufficient choice to ensure lexicographical order

$$R \prec \sum_{j=1}^{k} y_i \prec \sum_{j=1}^{k} y_j \lambda_j \tag{13}$$

is

$$\begin{cases} \beta > \gamma \sum_{j=1}^{k} \lambda_j \\ \\ \alpha > \beta(m-1) + \gamma \sum_{j=1}^{k} \lambda_j \end{cases} \tag{14}$$

where under the normalizzation $\sum_{j=1}^{k} \lambda_j = 1$ the easiest choise is

$$\begin{cases} \alpha = 2m \\ \beta = 2 \\ \gamma = 1 \end{cases} \tag{15}$$

This choice correctly implements the sequence of model optimizations. [3]

An other behavior could be

1. Minimize first number of activeted subsets

[3] Let be A and B two admissible solutions. Focus on R, if $R_A < R_B$ then at least $R_B - R_A \geq 1$, so in the worst scenario, solution A will be use more clique and have a much lower score. The maximum possible advantage for B is $\beta(m-1)+\gamma \sum_{j=1}^{k} \lambda_j$, so if $\alpha > \beta(m-1) + \gamma \sum_{j=1}^{k} \lambda_j$ no improvement in cardinality or score can compensate for a worsening of R. Therefore, the solver will always choose the minimum value of R first. Then, for the same R, if a solution uses one less clique, the minimum advantage in the $\beta \sum_{j=1}^{k} y_j$ term is β. The maximum possible advantage of the other solution in the score is $\gamma \sum_{j=1}^{k} \lambda_j$. Therefore, if $\beta > \gamma \sum_{j=1}^{k} \lambda_j$ the model will be prefer always less clique, indipendently from the score. Finaly, for the same R and number of activate cliques, it remains only the maximization of $-\gamma \sum_{j=1}^{k} y_j \lambda_j$.

2. Then, the minimization of maximum redundancy

3. Finally, maximize the consistency between the minimum subsets.

The lexicographical order is

$$\sum_{j=1}^{k} y_i \prec R \prec \sum_{j=1}^{k} y_j \lambda_j \tag{16}$$

the conditions will be

$$\begin{cases} \alpha > \gamma \sum_{j=1}^{k} \lambda_j \\ \\ \beta > \alpha R_{\max} + \gamma \sum_{j=1}^{k} \lambda_j \end{cases} \tag{17}$$

under the normalizzation $\sum_{j=1}^{k} \lambda_j = 1$ and the bound $R_{\max} \leq m$ the easiest choise is

$$\begin{cases} \alpha = 2 \\ \beta = 2m + 2 \\ \gamma = 1 \end{cases} \tag{18}$$

2 Conclusions

In this work I presented MaCliCo, an integer linear programming framework that builds a hypergraph from a weighted graph through maximal-clique-based set covering. The method enumerates all maximal cliques with the Bron–Kerbosch algorithm, scores each of them by its normalized internal cohesion, and selects the cliques that cover all vertices while balancing maximum redundancy, number of hyperedges and total cohesion. Also, they have been derived sufficient conditions on the parameters (α, β, γ) that guarantee a lexicographic priority among these objectives, together with explicit parameter choices for two representative behaviors. Future work will address the experimental evaluation of the framework and the scalability of the clique enumeration step on large graphs.